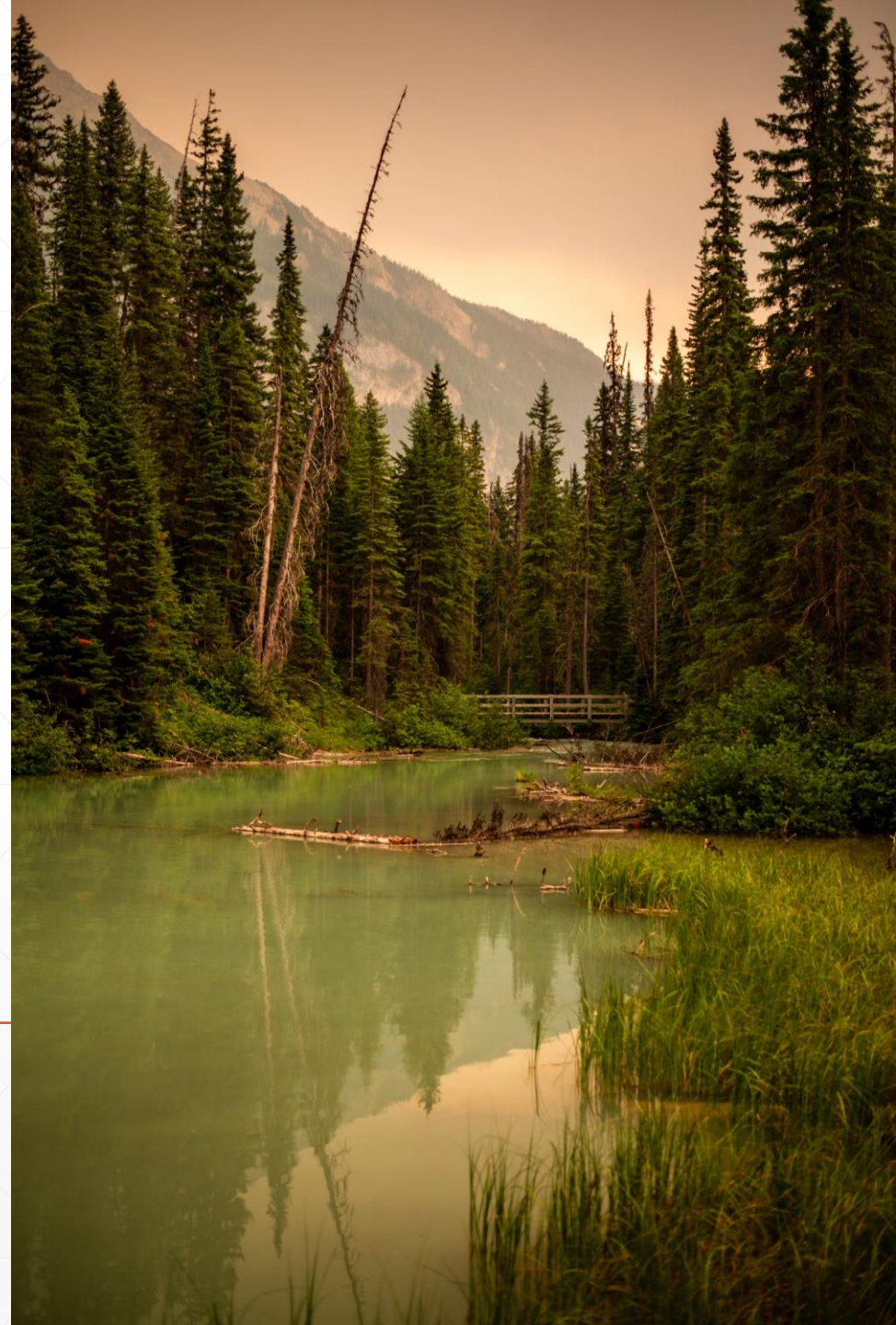


Netty多线程与异步编程模型

北京理工大学计算机学院
金旭亮





Netty线程模型

概述



众所周知，Netty被视为是一个高性能的网络开发框架。它的高性能，与其精心设计的线程模型是密切相关的。



Netty采用的是基于主从模式的Reactor线程模型，而其执行的I/O操作，也都是异步操作。



对于使用Netty开发网络应用的开发者来说，了解Netty的多线程与异步编程模型，是非常重要的。

EventLoop与EventLoopGroup



在Netty中，EventLoop可以看成是一个有着特定职责的线程函数，它监控Channel中的I/O事件，当事件触发时，通过回调ChannelHandler完成业务处理。



每个EventLoop都关联着一个线程，消息的读取、数据的编解码，以及后继的各个ChannelHandler，始终都由这个线程负责执行，由于全部是单线程，没有线程上下文切换，数据也就不存在被并发修改的危险。



每个EventLoop都关联着一个Selector，可以管理一到多个Channel，这些Channel都要向Selector进行注册，明确其感兴趣的I/O事件。当特定的I/O事件触发时，Selector会回调Channel所关联的Channel Pipeline中的ChannelHandler方法。



多个EventLoop归并为一个EventLoopGroup，因此，Netty中的EventLoopGroup可以看成是一个定制的线程池。

Netty的主从线程组模型



在Netty应用中，经常看到以下典型代码，里面创建了两个EventLoopGroup：

```
EventLoopGroup bossGroup = new NioEventLoopGroup();
EventLoopGroup workerGroup = new NioEventLoopGroup();

ServerBootstrap boot = new ServerBootstrap();

boot.group(bossGroup, workerGroup)
    .channel(NioServerSocketChannel.class)
    .childHandler(new ChannelInitializer<NioSocketChannel>() {
        @Override
        protected void initChannel(NioSocketChannel ch) throws Exception {
            //在此构建ChannelPipeline
        }
    });
```

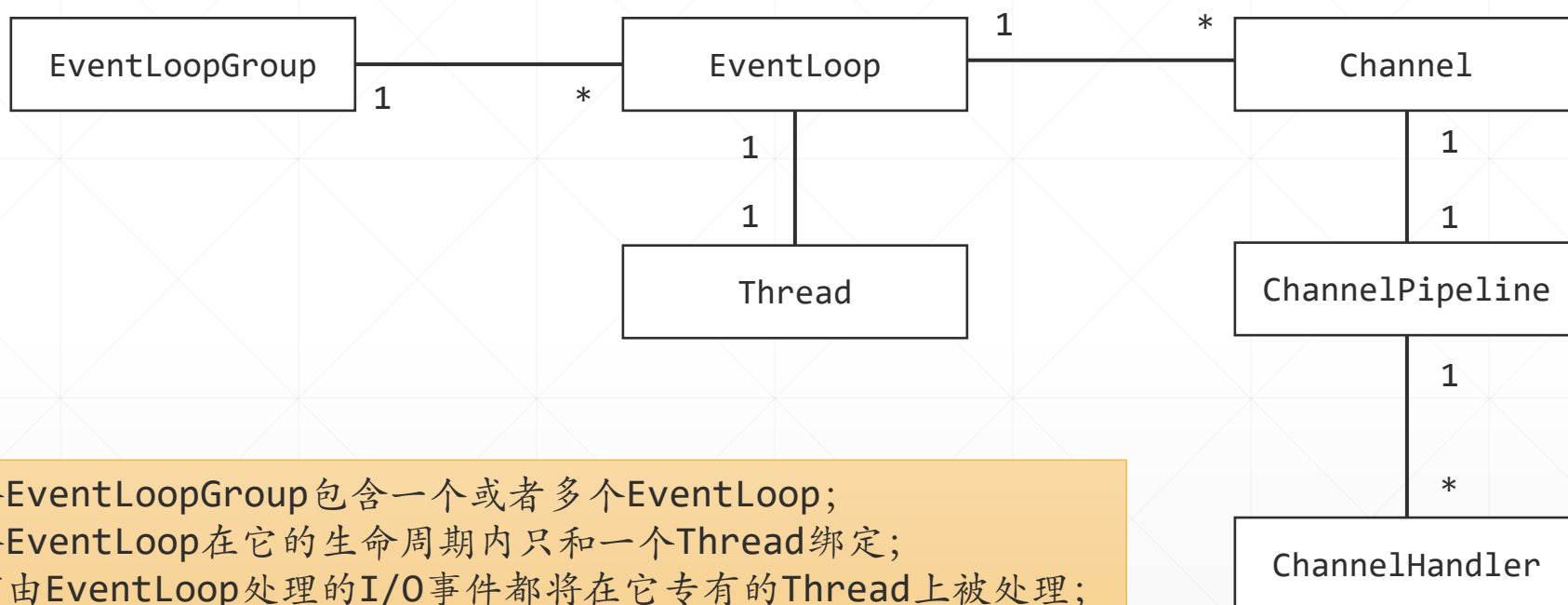


boss类型的EventLoopGroup()，负责接收客户端连接请求，它关联着一个Selector实例，并注册了一个监听指定端口的ServerSocketChannel实例。



当有连接到来，boss将创建SocketChannel转发给worker类型的EventLoopGroup，worker会从组中选择一个EventLoop，并把那个SocketChannel注册到EventLoop所关联的Selector中，并对其后的I/O事件进行处理（即调用管线中的ChannelHandler方法）。

Netty核心组件关联图



- 一个EventLoopGroup包含一个或者多个EventLoop;
- 一个EventLoop在它的生命周期内只和一个Thread绑定;
- 所有由EventLoop处理的I/O事件都将在它专有的Thread上被处理;
- 一个Channel在它的生命周期内只注册于一个EventLoop;
- 一个EventLoop可能会被分配给一个或多个Channel



Netty异步编程模型

Netty定制的Future<V>接口



Netty中，所有I/O操作都是异步的，为此，从JDK中的Future接口派生，定义了一个专用的Future接口，此接口从以下几个方面扩展的JDK中的Future接口：



操作完成的结果进行了区分，分为success、fail和canceled三种。



可以调用addListener()方法添加监听器对象。



定义有await()和sync()两个方法，以阻塞模式等待异步操作完成



定义有getNow方法获取当下异步操作的结果，如果异步操作还没完成，返回null。



在实际开发中经常用到的ChannelFuture，是Future<V>的子接口，它经常作为Netty组件方法的返回值类型，表示Channel的异步I/O操作的结果。

ChannelFuture接口



ChannelFuture提供了相应方法，获取异步操作的状态和结果，这些方法主要包含：

`isDone()`、`isSuccess()`、`isCancel()`和`cause()`



当异步操作失败时，`cause()`方法返回值非空，可以通过此方法的返回值了解发生了什么错误。



一个ChannelFuture要么是完成的，要么是未完成的。当一个I/O操作开始时，会创建一个ChannelFuture对象，等其操作完成，这个对象相应字段被赋予了相应的值。



ChannelFuture最常见的编程模式，就是给它添加一个监听器对象，然后在监听器所定义的方法，调用`isSuccess()`等方法检测操作是否成功，然后基于操作结果，进行相应的处理。

ChannelFuture使用示例

ChannelFuture定义了以下方法，用于添加监听器对象：

```
ChannelFuture addListener(GenericFutureListener<? extends Future<? super Void>> listener);
```

以下是一个示例：

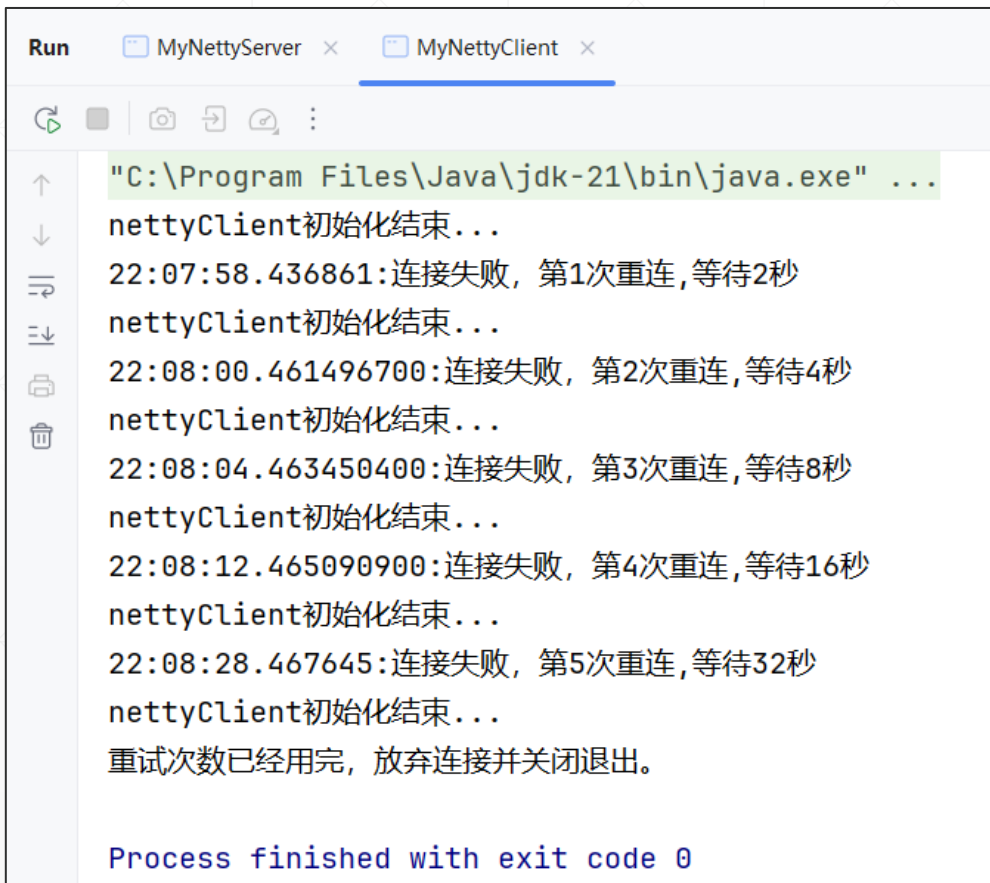
```
ChannelFuture future = serverBootstrap.bind(port).sync()
    .addListener(new GenericFutureListener<Future<? super Void>>() {
        @Override
        public void operationComplete(Future<? super Void> future) throws Exception {
            if(future.isSuccess()){
                System.out.println(port+"端口绑定成功.");
            }else {
                System.out.println(port+"端口绑定成功失败");
            }
        }
    });
```

自动重连.....

```
//最大尝试次数为5
private final static int MAX_RETRY = 5;
//自动重连5次
private static void connect(Bootstrap bootstrap, String host, int port, int retry) {
    bootstrap.connect(host, port).addListener(future -> {
        if (future.isSuccess()) {
            System.out.println("连接成功");
        } else {
            if (retry == 0) {
                System.out.println("重试次数已经用完, 放弃连接并关闭退出。");
                ((ChannelFuture) future).channel().close().sync();
                bootstrap.config().group().shutdownGracefully();
            } else {
                //这是第几次重连?
                int order = (MAX_RETRY - retry) + 1;
                //重新计算等待的时间
                int delayTime = 1 << order;
                System.out.println(LocalTime.now() + ":连接失败, 第" + order + "次重连, 等待" + delayTime + "秒");
                //调用EventLoopGroup的延时调用功能, 递归调用connect方法进行连接.....
                bootstrap.config().group().schedule(() ->
                    connect(bootstrap, host, port, retry - 1),
                    delayTime, TimeUnit.SECONDS);
            }
        }
    });
}
```

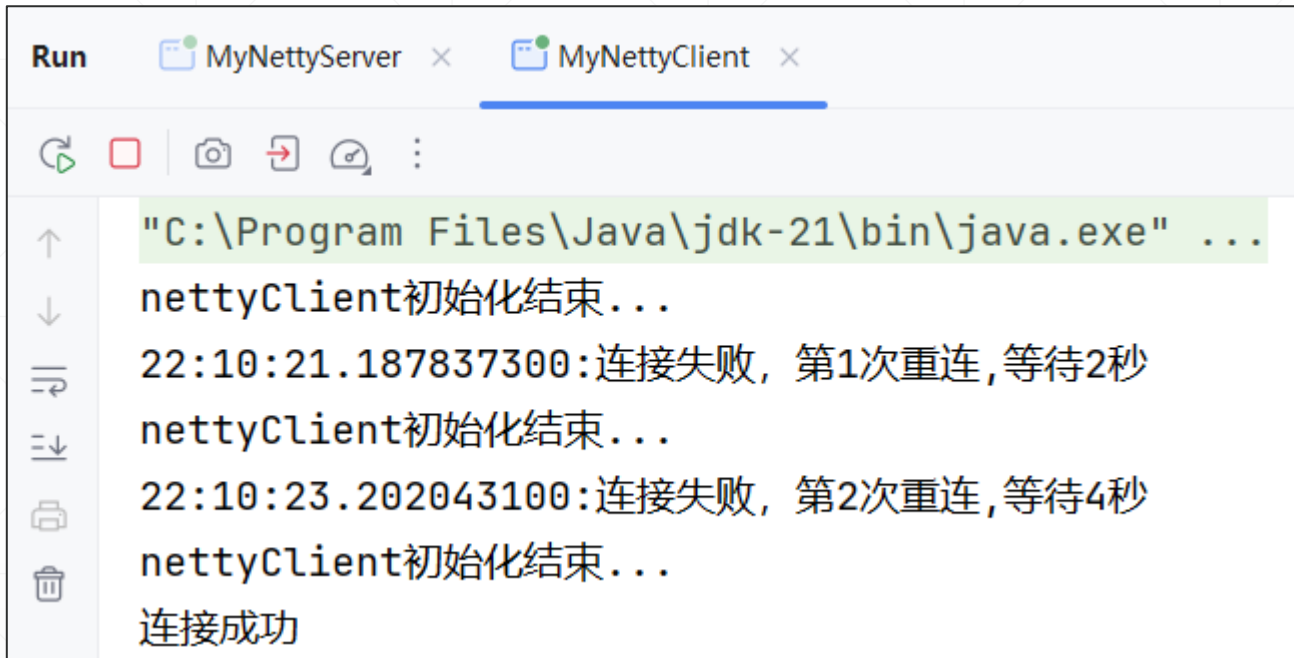
运行截图

如果服务端没有运行，则重试5次连接之后，客户端会自动退出：



```
Run  MyNettyServer x  MyNettyClient x
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
nettyClient初始化结束...
22:07:58.436861:连接失败, 第1次重连, 等待2秒
nettyClient初始化结束...
22:08:00.461496700:连接失败, 第2次重连, 等待4秒
nettyClient初始化结束...
22:08:04.463450400:连接失败, 第3次重连, 等待8秒
nettyClient初始化结束...
22:08:12.465090900:连接失败, 第4次重连, 等待16秒
nettyClient初始化结束...
22:08:28.467645:连接失败, 第5次重连, 等待32秒
nettyClient初始化结束...
重试次数已经用完, 放弃连接并关闭退出。

Process finished with exit code 0
```

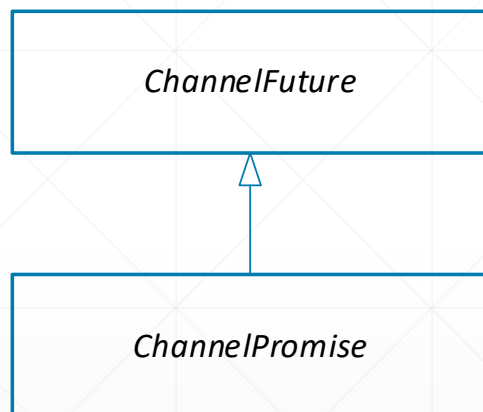


```
Run  MyNettyServer x  MyNettyClient x
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
nettyClient初始化结束...
22:10:21.187837300:连接失败, 第1次重连, 等待2秒
nettyClient初始化结束...
22:10:23.202043100:连接失败, 第2次重连, 等待4秒
nettyClient初始化结束...
连接成功
```

如果在客户端重试连接期间，服务端上线，则客户端会连接成功.....

ChannelPromise接口

许多出站Handler方法都需要一个ChannelPromise参数，以便在操作完成时得到通知。



`ChannelPromise` 衍生自 `ChannelFuture`, 被称为“可写的 `ChannelFuture`”，有 `setSuccess()` 和 `setFailure()` 等方法用于设置结果。

Netty异步编程模型小结



Netty异步编程模型中的核心类型就两个：ChannelFuture接口和ChannelPromise接口。



许多Netty组件（比如Channel）的方法，返回这两个接口的对象，或者是以这两个接口作为方法参数。



在实际开发中，比较常用的方式就是使用诸如sync()这样的方法阻塞等待异步操作的结束，或者是通过调用addListener()方法的方式给异步操作添加一个监听器，从而在异步操作完成时得到通知，查询异步操作的状态，取回异步操作的结果，或者是处理异步操作的错误。



在后面课程的例子中，会看到许多与ChannelFuture或ChannelPromise相关的代码，基本上就是上面介绍的两种编程模式，此处不再赘述。